AGENTGO · PHASE 1 REPORT · 2026-10-16

Status and roadmap

AgentGo is a Chromium browser for Windows that AI agents drive while a person watches and stays in charge. This report says what is finished, what was tested, which tools exist and what is still left, measured from the code on 2026-10-16.

1026
of 1039 register tools working, each with a test

25+5
integration test files (offline + live web)

157
unit tests passing

17/17
milestones done, 0 partly



own command 497 · sub-tool of a command 529 · impossible in the engine 13 · not built 0

Milestones

#	Milestone	Delivered	Left
0	**Repository, toolchain, contracts** Done	Rust workspace (16 crates), pinned CEF 152 / Chromium 152, protocol envelope, error codes, capability manifest, lockstep tests.	—
1	**CEF composition** Done	One window: trusted chrome, isolated content tabs (stacked full-size so background tabs stay usable), tools overlay; debug and release builds; portable package (the installer is still to do).	Code signing of the executable.
2	**Instances, profiles, tabs, product core** Done	Tabs, history, bookmarks, downloads, find bar, zoom, print, session restore, error page, Settings (startup, search engine, privacy, languages, downloads, system), durable profiles.	Several windows, tab groups and pinning.
3	**Universal protocol and isolation** Done	JSON lines over stdio, authenticated named pipe, CLI, MCP server; sessions, tab leases, pause and stop, idempotency, audit log.	—
4	**Human-equivalent control** Done	Click, type, fill, select, drag and drop, private clipboard, touch, pen, held buttons and keys, upload; agent's own pointer and keyboard, never the person's.	IME composition events; driving native file dialogs.
5	**Compact observation and grounding** Done	Outline with references and change sets (≈56× smaller than HTML), locators, grounding by description with confidence, accessibility views, screenshots, frame recording, DOM snapshot.	OCR, visual grounding, visual diff, cross-origin frames.

6	**Batch workflows** Done	Batches, search to records, collect, fill forms, workflow record and replay with healing, decision record/apply for Laya.	Agent task engine (plan, queue, parallel tabs, checkpoints of a task).
7	**Policy, secrets, approvals, handoff** Done	Approval and full modes, permanent purchase stop, CAPTCHA and login handoff (never solved), DPAPI sign-in vault with autofill the agent never sees, save prompt, Account Center with blocking, domain allow and block lists, site permissions granted by the person.	—
8	**Artifacts, downloads, Windows lane** Done	Artifacts (list, read, hash, redact, bundle, retention), print to PDF, downloads with progress, open and reveal after approval, scoped file operations. Native Windows UI Automation lane (29 tools, tested on WPF and WinForms windows). Every finished download gets Mark of the Web and a Microsoft Defender scan; a flagged file cannot be opened.	Scanning needs Microsoft Defender (other antivirus products are not called; without Defender the result is 'unavailable'); a real malware sample was not used in tests.
9	**Skills and adapters** Done	Bundled skill and generated command reference, MCP tools including browser_guide and browser_account, CLI.	SDK packages for Rust, TypeScript and Python; skill management commands.
10	**Daily-browser P1/P2** Done	Chrome Web Store extensions (content scripts, toolbar buttons), folder sync, sites as Start-menu apps, DevTools, permissions, find, zoom, session restore, several windows, tab groups and pinning (Menu: pin, group, remove from group), address-bar suggestions, Translate page and Reader view in the Menu.	Translate uses Google's page proxy; Reader view is plain text (no images); no drag-to-group or tab context menu; groups cannot be renamed from the UI.
11	**Performance, security, release** Done	Release build, portable zip with checksums (installer still to do), licence audit, secret scan, protocol fuzz test, performance benchmark, metrics and benchmark commands.	Code signing and auto-update; the BrowserGym evaluation (next phase).
12	**Quiet background work and accounts** Done	Window opens behind yours and never takes focus, agent pointer and typing animations, Account Center with OAuth API connectors, saved sign-ins, Laya interface and setting.	Laya model itself (training is the next phase).
13	**Privacy, VPN and Settings search** Done	Built-in ad and tracker blocker (Brave adblock engine, EasyList and EasyPrivacy bundled, on by default, operator-only switch, live counter). VPN through Cloudflare WARP (usque sidecar, MIT; off by default, operator-only, covers every tab and side window, WebRTC leak policy, fails closed, sidecar dies with the browser). Animated Settings search with typo tolerance, synonyms and related settings. Each has offline and live tests.	Installer (portable zip only for now); VPN uses an unofficial WARP client and accepts Cloudflare terms for the person; no per-site VPN or country choice; Tor was tried (arti) and dropped, its bootstrap hung; EasyList refresh is a manual script.

14	**Everyday browsing polish** Done	Favorites bar under the address bar with an animated star (add, reorder by drag, middle-click, remove, overflow menu, Ctrl+Shift+B). Extensions get toolbar buttons (pin or hide each one) and an Extensions panel (turn on or off, remove, Chrome Web Store link). Open panels slide away when the page is clicked. A Toolbar section in Settings (favorites bar, Home button, extension buttons). The window now comes to the front when the person launches it.	Installer; extension pop-ups are not supported; page-click and foreground behaviour are tested with synthetic messages on a private desktop, not on a real one; sdk_e2e needs cargo on PATH.				
15	**Stronger blocker, smoother panels** Done	Ad blocker now ships 13 lists (EasyList, EasyPrivacy, uBlock Origin, AdGuard, Peter Lowe, URLhaus, HaGeZi Pro) precompiled into an engine snapshot, with cosmetic hiding (generic and per-site), scriptlets and redirect resources: 62% to 98% on turtlecute.org (129 of 132; the 3 left are rules for that one site), 99% on superadblocktest.com (478 of 482), 100/100 on adblock-tester.com; cookie-banner lists and AgentGo's own host list added; requests wait for the engine at launch instead of passing unfiltered. Panels close with a time-based animation at a 1 ms timer resolution and no blur filters. Alt+Tab and taskbar clicks by the person bring AgentGo forward; agents still cannot.	The few hosts still reachable are on purpose (redirector.googlevideo.com would break video; some are replaced by harmless stand-ins); Alt+Tab and taskbar behaviour is not verified on a real desktop; list data is GPLv3 (see third_party/README.md); list refresh is a manual script.				
17	Human browser chrome and product features		63		23	40	0 0

How it was verified

- **Unit tests** (Rust): protocol, policy, accounts and PKCE attack cases, vault, decision records, recorder, extension patterns, packaging helpers — 157 passing.
- **Integration tests** drive the real browser on a hidden desktop: accounts_e2e, adapters_e2e, adblock_e2e, agent_e2e, browser_e2e, capabilities_e2e, daily_e2e, favorites_e2e, native_e2e, persistence_e2e, presentation_e2e, profiles_e2e, protocol_smoke, rest_e2e, safety_reader_e2e, sdk_e2e, sessions_e2e, settings_search_e2e, shell_e2e, site_signals_e2e, suite_e2e, tasks_e2e, tools_e2e, ui_surface_smoke, windows_e2e. Every one of the 1026 working register tools is called with real parameters and its answer checked.
- **Live web**: adblock_live_e2e, real_web_e2e, site_render_smoke, store_e2e, vpn_live_e2e — Wikipedia, Hacker News, a shop, a real form, dynamic pages, dialogs, drag and drop, GitHub, and an extension installed from the Chrome Web Store (Dark Reader). Blind runs on unfamiliar sites (python.org, a demo shop, a to-do app) found five outline faults, all fixed with regression checks.
- **Release package**: the same suite passed against the packaged release build, run from a folder outside the repository.

- **Speed** (debug build): start 585 ms, observe 12 ms, click 395 ms, screenshot 159 ms.

Tools by domain

#	Domain	Tools	Own	Sub-tool	Impossible	Left
1	Browser runtime and process lifecycle	26	21	3	2	0
2	Profiles, user contexts, and session state	30	26	4	0	0
3	Windows, browser windows, and viewport control	30	19	11	0	0
4	Tabs, frames, browsing contexts, and popups	46	23	23	0	0
5	Navigation, loading, and synchronization	35	16	19	0	0
6	Human-equivalent mouse, pointer, touch, and gesture input	63	36	27	0	0
7	Clipboard, selection, focus, and editing	32	5	27	0	0
8	Semantic page observation and visual grounding	55	22	33	0	0
9	Locators, grounding, and target resolution	36	1	35	0	0
10	Page actions and form operations	56	21	35	0	0
11	JavaScript, DOM, and page-runtime control	21	8	13	0	0
12	Files, native dialogs, downloads, and printing	40	30	10	0	0

13	Browser dialogs, menus, prompts, and permissions	27	12	15	0	0
14	Storage, cookies, credentials, and site data	29	6	23	0	0
15	Network, requests, responses, and web platform events	35	21	14	0	0
16	Media, camera, microphone, sensors, and devices	30	4	21	5	0
18	Extensions, scripts, and Agentgo plugins	19	14	0	5	0
19	Agent one-call execution and workflow control	42	26	16	0	0
20	Batch execution, synchronization, and recovery	28	12	16	0	0
21	Research, extraction, and data workflows	35	6	29	0	0
22	Skills, memory, replay, and domain adapters	35	28	7	0	0
23	Approval modes, risk policy, and human handoff	29	10	19	0	0
24	Human handoff UI and Agentgo shell functions	38	21	17	0	0
25	Artifacts, evidence, and context control	32	15	17	0	0
26	Native Windows control lane	32	31	1	0	0
27	Accessibility and inclusive interaction	21	1	20	0	0

28	Debugging, inspection, and diagnostics	24	7	16	1	0
29	Metrics and evaluation	24	23	1	0	0
30	Agent control protocol and integration surface	26	9	17	0	0
	Total	**1039**	**497**	**529**	**13**	**0**
17	**Release polish** Done	Command palette (Ctrl+K, fuzzy search over settings, tabs, history, bookmarks and actions). First-run welcome with import of history, bookmarks and open tabs from Chrome, Edge, Brave and Firefox. Per-site dark mode (own switch directly under Theme, brightness and contrast sliders, excluded-site list). Site permissions stay blocked by default and ask in a themed box (Keep blocked, Allow this time, Always allow); allowed sites are listed in Settings and can be blocked again. Translation: selection card and whole-page translate bar with a language picker and a "Translate into" setting. Switches animate in place without redrawing Settings. Report a problem (zip of diagnostics, nothing sent). Print to PDF. Prompt-injection scanner for page text given to agents. New Tab page: an animated film of an agent reading a page, filling a form, asking for approval and finishing. The VPN was removed (card shows "coming soon").	"Ask your agent" in the problem report only copies text; no GitHub issue link until a real repository address is set in `Cargo.toml`; Firefox passwords and Chrome v20-encrypted passwords are not imported; translation uses Google's service; the executable is not Authenticode-signed.				

A sub-tool is a register name that runs one of the browser's commands with the same effect (for example `browser_ui.back` runs `tab.back`); it is called and checked by the tests like any command. Every name not built yet answers `capability.unsupported` with "not built yet", so an agent is never left guessing.

What is left to build (0 tools)

#	Domain and tools	Count

What cannot be done in this browser (13)

Tool	Why
`browser.engine_switch`	AgentGo is built on one engine (Chromium through CEF) and cannot switch engines.

`browser.engine_matrix`	AgentGo is built on one engine (Chromium through CEF).
`device.bluetooth`	pages are not given access to real hardware devices.
`device.usb`	pages are not given access to real hardware devices.
`device.serial`	pages are not given access to real hardware devices.
`device.hid`	pages are not given access to real hardware devices.
`device.nfc`	Web NFC exists only on Android.
`extension.permission_request`	AgentGo runs the content scripts of extensions (extension.install); the chrome.* extension APIs for toolbar buttons, menus and native messaging are not part of the embedded engine.
`extension.browser_actions`	AgentGo runs the content scripts of extensions (extension.install); the chrome.* extension APIs for toolbar buttons, menus and native messaging are not part of the embedded engine.
`extension.context_menu`	AgentGo runs the content scripts of extensions (extension.install); the chrome.* extension APIs for toolbar buttons, menus and native messaging are not part of the embedded engine.
`native_messaging.list`	AgentGo runs the content scripts of extensions (extension.install); the chrome.* extension APIs for toolbar buttons, menus and native messaging are not part of the embedded engine.
`native_messaging.invoke`	AgentGo runs the content scripts of extensions (extension.install); the chrome.* extension APIs for toolbar buttons, menus and native messaging are not part of the embedded engine.
`debug.breakpoint`	a page paused at a breakpoint also stops the agent's own control of it; open the DevTools window (debug.devtools_open) to debug.

Known limits and open points

- Google shows a CAPTCHA to brand-new profiles after many automated runs. AgentGo hands it to the person (by design, never solved); the person's real profile, or Bing/DuckDuckGo in Settings, avoids most of them.
- Extensions run their content scripts only; toolbar buttons, pop-ups, background workers and native messaging of Chrome extensions are not part of the embedded engine.
- OAuth API connectors were tested end to end against a mock provider (PKCE, refresh, forged and replayed answers). Real Google, GitHub and Yandex sign-in needs the person's own OAuth client id and was not tried live.
- Camera, microphone and location now ask the person; notifications are still refused by the embedded engine even when granted.
- Some older end-to-end files (`ui_surface_smoke`, `windows_e2e`, `rest_e2e`) were not re-run for this release; the Rust suite and the files listed in the changelog were.
- The VPN was removed from this release; the Settings card says "coming soon".
- The ad blocker bundles EasyList (CC BY-SA 3.0 / GPLv3) as data; notices ship in `licenses/`.
- The executable is not code-signed; Windows SmartScreen may ask once. There is no auto-update.
- Every open tab keeps rendering so agents can work in background tabs; very many tabs cost memory.

Next phase: Laya and the experiment

The browser is ready for the graduation experiment (configurations A, B, C0, C1). What is already in place for it:

- **Decision record** (`decision.record` / `decision.apply`, schema `agentgo.laya.decision/0`): the subgoal, compact state, every possible step as a candidate, supplied values and recent outcomes — exactly what Laya will be given. Documented in `skills/agentgo/references/laya.md`.
- **Training data**: `workflow.record` writes every step, including full decision records and their outcomes, as JSON lines.
- **Measurement**: `metrics.*` (latency, errors, recoveries, handoffs, reply tokens, task success, tokens per success, turns) and `benchmark.*` (record, compare, regression, replay a recording N times).
- **Setting**: "Laya, the local choice model" in Settings (off by default; your own model decides when it is off).

Still to do: build the local worker that loads a Laya checkpoint and answers decision records; collect trajectories; fine-tune; run the A/B/C0/C1 comparison (BrowserGym tasks plus AgentGo fixtures).

Starting again

- Read `README.md`, then `docs/HANDOFF-OPUS.md` and `docs/issues-log.md` (every problem met and how it was solved).
- Build: `powershell -File scripts\build.ps1 -Runtime` (CEF is fetched by `scripts\bootstrap.ps1` from `scripts\pins.json`). Test: `node scripts\dev\run-integration.js`. Package: `scripts\package.ps1 -Release`.
- Add a tool: one line in `scripts/dev/tools-spec.js` (plus a function in `tools.js` if it reads the page), then `gen-tools.js`, `gen-skill.js`, build, `tools_e2e`.